HyperAD

4-Person Team Plan — Task Division & Workflow

Person	Code	Role	Domain
Person 1	**P1**	Algorithms & Graph Engine	Bellman-Ford · Tarjan · HITS · Frigioni · BFS · Hypergraph
Person 2	**P2**	Data Collection Layer	LDAP · Kerberos · ACL Crawler · Session Map · Neo4j
Person 3	**P3**	AI Agent & Reporting	LangChain · MITRE Map · Confidence · PDF Report · Paper
Person 4	**P4**	Frontend & DevOps	React · D3.js · FastAPI · Docker · GitHub · Demo

The diagram on the next page shows the full task flow. Tasks placed SIDE-BY-SIDE in the same row run in PARALLEL — all four people work simultaneously. Tasks stacked TOP-TO-BOTTOM are SERIAL — the lower one cannot start until the upper one is done. Red SYNC bars are hard blockers: all work stops, integration happens, then the next parallel phase begins.

There are 2 internal sync points and 1 final sync. Outside of those, all four streams are fully independent — meaning 75% of the project can be built simultaneously. This is the critical path: P1 finishing the algorithm modules unlocks P3's AI agent. P2 finishing the collector unlocks P1's graph builder. Everything else is parallel.

P1	P2	P3	P4
Algorithms & Graph	Data Collection	AI Agent & Report	Frontend / Dashboard

ALL HANDS PHASE 0 — Lab Setup | Week 1

SYNC — Everyone: Install VirtualBox, Windows Server DC, Kali, Neo4j, Git, Python

P1	P2	P3	P4
Temporal Bellman-Ford algorithms/temporal_bf.py	**LDAP Enumerator** collectors/ldap_enum.py	**LangChain Agent Setup** ai_agent/agent_core.py	**React App Bootstrap** dashboard/ + FastAPI skeleton
Tarjan SCC algorithms/tarjan_scc.py	**Kerberos Scanner** collectors/kerberos_scan.py	**MITRE Mapper** ai_agent/mitre_mapper.py	**D3.js Graph Explorer** GraphExplorer.jsx
HITS Scorer algorithms/hits_scorer.py	**ACL Crawler** collectors/acl_crawler.py	**Confidence Scorer** ai_agent/confidence_scorer.py	**Blast Radius UI** BlastRadius.jsx
Bidirectional BFS algorithms/blast_radius.py	**Session + Trust Mapper** collectors/session_trust.py	**PDF Report Generator** reporting/report_gen.py	**Findings Panel UI** FindingsList.jsx

SYNC — SYNC 1 — P2 hands snapshot JSON to P1 + P3. P1 hands algorithm modules to P3. P4 needs API from P1.

P1	P2	P3	P4
Hypergraph Builder graph_engine/hg_builder.py	**Edge Scorer (decay)** graph_engine/edge_scorer.py	**Counterfactual Engine** ai_agent/counterfactual.py	**Remediation Preview UI** FixPreview.jsx
Frigioni DSSSP algorithms/dynamic_sssp.py	**Neo4j Graph Store** graph_engine/graph_store.py	**Delta Alert Reporter** reporting/delta_reporter.py	**FastAPI /endpoints** api/main.py

SYNC — SYNC 2 — Full pipeline connected: Collector → Graph → Algorithms → Agent → Report + UI. End-to-end test.

P1	P2	P3	P4
Algorithm Unit Tests pytest + hypothesis	**Lab Scenario Setup** 50/200/500 user AD envs	**Paper — Sec I–V** Intro, BG, Architecture	**Docker Compose Setup** docker-compose.yml
Complexity Analysis Paper Sec VI — Algorithms	**BloodHound Comparison** Run BH + PingCastle on labs	**Paper — Sec VII–X** Eval, Discussion, Conclusion	**GitHub README + Demo** README.md + demo GIF

SYNC — FINAL — Paper reviewed by all 4. GitHub release. IEEE submission.

Detailed Task Breakdown by Person

Each person's tasks are listed below in the order they should be built. Dependencies on other people's work are explicitly called out.

P1 — Person 1 — Algorithms & Graph Engine

PHASE 0 — Week 1 | SERIAL

Task / Module	What to build
Lab setup	Install Kali, Python, Neo4j, NetworkX, HyperNetX. Clone repo. Verify Neo4j browser at localhost:7474.

PHASE 1 — Weeks 2–4 | PARALLEL with P2, P3, P4

Task / Module	What to build
temporal_bellman_ford.py	Implement Bellman-Ford with temporal decay edge weights. Input: NetworkX DiGraph. Output: list of (path, risk_score) tuples sorted descending.
tarjan_scc.py	Implement Tarjan's SCC on the delegation subgraph. Flag any SCC with size > 1 as Critical finding. Input: directed graph. Output: list of {nodes, size, risk_level}.
hits_scorer.py	Run nx.hits() on full AD graph. Return top-20 nodes by authority score. These become priority findings.
blast_radius.py	Bidirectional BFS from a compromised_node parameter. Forward BFS from node, backward BFS from Domain Admin nodes. Output: reachable subgraph + hop distances.
Unit tests	Write tests/test_p1_algorithms.py with a 12-node synthetic graph. Assert: BF finds all known paths, Tarjan catches the planted cycle, HITS ranks the hub node highest.

BLOCKER — SYNC 1 — End of Week 6 | BLOCKER

Task / Module	What to build
Hand off to P3	Export all algorithm modules as importable Python functions. P3 registers these as LangChain tools.
Hand off to P4	Expose blast_radius() and get_paths() via FastAPI endpoints. P4 calls these from the dashboard.
Receive from P2	Receive snapshot JSON from P2. Use it as input to test graph builder on real data.

PHASE 2 — Weeks 7–9 | PARALLEL with P2, P3, P4

Task / Module	What to build
hypergraph_builder.py	Ingest P2's snapshot JSON. Build NetworkX DiGraph + HyperNetX for multi-entity edges (GPO → many OUs). Add temporal decay weights via P2's edge_scorer output.
dynamic_sssp.py	Frigioni incremental SSSP. Store baseline shortest-path trees. On delta input (list of changed edges), update only affected trees. Log speedup vs full recomputation.

Task / Module	What to build
graph_store.py collaboration	Work with P2 to finalise Neo4j schema. Define MERGE Cypher patterns for all node/edge types.

PHASE 3 — Weeks 10–14 | PARALLEL with P2, P3, P4

Task / Module	What to build
pytest + hypothesis	Property-based tests for graph algorithms. Test on 50/200/500 node graphs. Measure and record wall-clock time for Bellman-Ford and Frigioni on each size.
Paper Section VI	Write algorithm pseudocode and complexity proofs for the IEEE paper. Provide to P3 who is leading paper writing.
Performance benchmarks	Generate Table 1 for paper: runtime comparison Frigioni vs naive across edge-change counts (10/50/100/500). Use timeit module.

P2 — Person 2 — Data Collection Layer

PHASE 0 — Week 1 | SERIAL

Task / Module	What to build
Lab AD setup	Create the Windows Server 2022 DC. Domain: lab.local. Create 25 users, 6 groups, 4 SPNs, 3 misconfigs: unconstrained delegation on svc_sql, AS-REP on testuser, GenericAll on IT_Admins group.

PHASE 1 — Weeks 2–5 | PARALLEL with P1, P3, P4

Task / Module	What to build
ldap_enum.py	Bind to DC via ldap3. Pull all users, groups, computers, OUs, GPOs, ACEs. Attribute list: sAMAccountName, memberOf, servicePrincipalName, msDS-AllowedToDelegateTo, userAccountControl, pwdLastSet, lastLogon. Timestamp every attribute.
kerberos_scan.py	Use Impacket GetUserSPNs to find Kerberoastable accounts. Use GetNPUsers for AS-REP roastable. Store SPN strings and ticket hashes.
acl_crawler.py	Parse nTSecurityDescriptor for every object. Extract each ACE: trustee SID, access mask, inheritance. Build SID-to-name cache. Output permission tuples.
session_mapper.py + trust_mapper.py	SMB NetSessionEnum for active sessions. Enumerate domain/forest trusts via LDAP trustType attribute.
Snapshot output	Write data/snapshot_YYYYMMDD_HHMMSS.json with every attribute wrapped as {value, last_seen, first_seen}. Test: verify all 3 planted misconfigs appear in JSON.

BLOCKER — SYNC 1 — End of Week 6 | BLOCKER

Task / Module	What to build
Hand off snapshot to P1	Provide snapshot JSON. P1 uses it for hypergraph_builder.py.
Hand off snapshot to P3	P3 needs it to test the AI agent's query_graph tool.

Task / Module	What to build
Receive schema from P1	P1 defines Neo4j node/edge schema. P2 implements graph_store.py accordingly.

PHASE 2 — Weeks 7–9 | PARALLEL with P1, P3, P4

Task / Module	What to build
edge_scorer.py	Implement temporal decay: w = base_risk * exp(-0.02 * days_since_last_use). base_risk table: GenericAll=10, WriteDACL=8, DCSync=10, Delegation=7, MemberOf=2, Session=3. Output: same graph with weight attributes added.
graph_store.py	Push all nodes and edges to Neo4j using Cypher MERGE. Tag every node with AD object type. Verify in Neo4j browser that the 3 misconfigs form visible paths to Domain Admin.
Snapshot manager	snapshot_manager.py stores timestamped snapshots. graph_differ.py computes delta between two snapshots (added/removed nodes and edges list).

PHASE 3 — Weeks 10–14 | PARALLEL with P1, P3, P4

Task / Module	What to build
Lab scenarios	Create 3 AD lab snapshots: Small (50 users, 5 misconfigs), Medium (200 users, 15 misconfigs), Large (500 users, 30 misconfigs) using PowerShell AD module scripts.
Comparative testing	Run BloodHound and PingCastle against the same lab environments. Record: findings count, runtime, false positives. These are the baseline numbers for the paper's evaluation section.
Paper Section VII data	Provide P3 with all evaluation numbers, lab environment descriptions, and collector methodology for the paper's Implementation section.

P3 — Person 3 — AI Agent & Reporting

PHASE 0 — Week 1 | SERIAL

Task / Module	What to build
Environment setup	Install Ollama + pull llama3:8b (or set up OpenAI API key). Install LangChain, ReportLab, python-docx. Verify Ollama responds to a test prompt.

PHASE 1 — Weeks 2–6 | PARALLEL with P1, P2, P4

Task / Module	What to build
agent_core.py skeleton	Set up LangChain ReAct agent. Register 4 placeholder tools: query_graph, get_paths, blast_radius, get_findings. Test with mock returns before real algorithms are ready.
mitre_mapper.py	Build the finding-type to ATT&CK; tactic lookup table. Kerberoastable SPN → T1558.003. Unconstrained delegation → T1134.001. DCSync rights → T1003.006. GenericAll on DA → T1098. AS-REP → T1558.004. At least 15 mappings.
confidence_scorer.py	Implement Bayesian scoring: P = P_active x P_weak_pwd x P_reachable. P_active = sigmoid((365 - days_since_login) / 100). P_weak_pwd = 1 if no pwdLastSet else sigmoid((180 - pwd_age) / 60).

Task / Module	What to build
report_generator.py skeleton	Build ReportLab PDF structure: cover, executive summary, findings section template, appendix. Use placeholder data until real findings available post-Sync 1.

BLOCKER — SYNC 1 — End of Week 6 | BLOCKER

Task / Module	What to build
Receive algorithm modules from P1	Wire real P1 functions into agent tool registrations. Replace mock tool returns with actual Bellman-Ford, Tarjan, HITS, BFS outputs.
Receive snapshot from P2	Test agent's query_graph tool against real Neo4j data. Verify MITRE mappings fire on the 3 planted misconfigs.

PHASE 2 — Weeks 7–9 | PARALLEL with P1, P2, P4

Task / Module	What to build
counterfactual.py	Agent tool: remove a finding's edge from a copy of the graph, re-run Bellman-Ford, return paths_before and paths_after counts. Wired to P4's Fix Preview UI via API.
delta_reporter.py	Generate a diff-based alert PDF: compare two snapshot findings lists, report only new findings since last run. Used by the continuous monitoring loop.
Full report_generator.py	Complete the PDF report with real findings data: per-finding pages with MITRE tactic, confidence score, Bellman-Ford path diagram (drawn with ReportLab graphics), remediation steps.

PHASE 3 — Weeks 10–14 | PARALLEL with P1, P2, P4

Task / Module	What to build
IEEE paper — Sections I–V	Write Abstract, Introduction, Related Work (cite BloodHound, PingCastle, Frigioni 2000, Kleinberg 1999, Berge 1973), System Architecture, Theoretical Model.
IEEE paper — Sections VI–X	Receive pseudocode from P1 (Sec VI). Receive evaluation numbers from P2 (Sec VIII). Write Implementation (VII), Discussion (IX), Conclusion (X). Compile full paper in IEEE LaTeX template.
Paper review coordination	Share draft with P1 and P2 for technical accuracy check on algorithm descriptions and evaluation data. Incorporate feedback. Submit.

P4 — Person 4 — Frontend & DevOps

PHASE 0 — Week 1 | SERIAL

Task / Module	What to build
Environment setup	Install Node.js 20+, npx, Docker Desktop. Bootstrap React app: npx create-react-app hyperAD-ui --template typescript. Install D3.js v7, axios, recharts.

PHASE 1 — Weeks 2–6 | PARALLEL with P1, P2, P3

Task / Module	What to build
FastAPI skeleton — api/main.py	Set up FastAPI app with placeholder routes: GET /findings, GET /graph, POST /blast-radius/{node}, POST /counterfactual/{finding_id}. Returns mock data. P1 and P3 will fill real logic later.
GraphExplorer.jsx	D3.js forceSimulation graph component. Nodes coloured by type: user=blue, group=purple, computer=grey, serviceAccount=amber. Edge thickness proportional to risk weight. Clicking a node calls sendPrompt() / triggers blast radius.
FindingsList.jsx	Findings panel sorted by risk score. Each card shows: finding title, MITRE tactic badge, confidence %, severity pill (Critical/High/Medium/Low), expand to see evidence + remediation.
BlastRadius.jsx	Heatmap overlay component. Given a node ID, fetch /blast-radius, overlay coloured rings: 1-hop=red, 2-hop=amber, 3+=yellow. Domain Admin reachable = pulsing red border via CSS animation.

BLOCKER — SYNC 1 — End of Week 6 | BLOCKER

Task / Module	What to build
Wire FastAPI to real modules	Replace mock returns in api/main.py with calls to P1 algorithm functions and P3 agent. Test each endpoint returns real data.
Frontend integration test	Connect GraphExplorer.jsx to GET /graph. Verify real Neo4j data renders correctly. Fix any CORS or data shape issues.

PHASE 2 — Weeks 7–9 | PARALLEL with P1, P2, P3

Task / Module	What to build
FixPreview.jsx — counterfactual UI	Each finding card gets a 'What if we fix this?' button. On click: POST /counterfactual/{id}, show before/after path count side-by-side with a delta badge.
docker-compose.yml	Define services: neo4j (ports 7474/7687), api (FastAPI, port 8000), ui (React, port 3000). Add environment variable config for credentials. Test: docker compose up --build should start all 3.
WebSocket live updates	api/main.py WebSocket endpoint /ws/alerts. When delta_reporter fires a new finding, push it to connected clients. Frontend shows a toast notification.

PHASE 3 — Weeks 10–14 | PARALLEL with P1, P2, P3

Task / Module	What to build
GitHub repository polish	Full README.md: project overview, architecture diagram (link to PDF), quick-start (docker compose up), lab setup instructions, screenshots.
Demo recording	Record a 3-minute demo video: collector running against lab DC → graph populating in Neo4j → AI agent finding Kerberoasting → PDF report generated → dashboard showing blast radius. Upload to GitHub.
Paper figures	Generate all figures for the IEEE paper: system architecture diagram (export from dashboard), algorithm complexity comparison chart, evaluation result graphs. Provide to P3.

Critical Path & Sync Point Detail

Two things can kill a group project: people blocking each other without knowing it, and people integrating incompatible code on the last day. The two sync points below are explicitly designed to prevent both. Every person must stop their parallel stream, run the integration steps, and confirm the pipeline works before continuing.

SYNC 1 — End of Week 6

This is the most critical sync. Three handoffs happen simultaneously:

Handoff / Action	Detail
P2 → P1 + P3	P2 delivers snapshot_.json. P1 feeds it into hypergraph_builder.py. P3 uses it to test agent query tools against real data.
P1 → P3	P1 delivers importable algorithm modules. P3 replaces mock tool returns in agent_core.py with real function calls.
P1 → P4	P1 exposes blast_radius() and get_paths() as FastAPI endpoints. P4 wires GraphExplorer.jsx to real data.
Integration test	Run the full pipeline once: collector → graph → algorithms → agent → PDF. At least one Critical finding must appear for the 3 planted misconfigs. If not, do not proceed to Phase 2.

SYNC 2 — End of Week 9

Full system integration — every component connected end-to-end:

Handoff / Action	Detail
P3 → P4	P3's PDF report and delta alert system are live. P4 wires the dashboard's alert toast to the WebSocket endpoint.
P1 → P2	Frigioni DSSSP is integrated with P2's graph_differ output. Test incremental update on a real delta.
End-to-end test	Run overnight simulation: take snapshot, inject a new dangerous permission via PowerShell, run delta engine, verify alert fires and new attack path appears in dashboard within 1 run cycle.
Freeze codebase	After Sync 2, no new features. Only bug fixes. P3 begins paper writing with complete, stable system data.

FINAL — Week 14

Paper and release:

Handoff / Action	Detail
Paper review	P3 shares full draft. P1 verifies algorithm pseudocode accuracy. P2 verifies evaluation methodology and numbers. P4 provides all figures.
GitHub release	Tag v1.0.0. Verify docker compose up works from a fresh clone on a machine with no prior setup. Test README instructions.
IEEE submission	Proofread abstract and conclusion. Check citation format. Submit to target venue (IEEE Access recommended for first submission).

What Can and Cannot Be Parallelised — Reference Table

Task	Parallel?	Blocked by	Reason
P1: Bellman-Ford, Tarjan, HITS, BFS	YES	Nothing	Pure algorithms, no external input needed
P2: LDAP, Kerberos, ACL, Session collectors	YES	Nothing	Just needs network access to lab DC
P3: Agent skeleton, MITRE mapper, ConfidenceScorer	YES	Nothing	Can use mock data until Sync 1
P4: React UI, D3 graph, FindingsList	YES	Nothing	Can render mock JSON from day 1
P1: Hypergraph builder	NO	P2 snapshot JSON	Needs real AD data as input
P1: Frigioni DSSSP	NO	P2 graph_differ output	Needs delta between two real snapshots
P3: Agent tool wiring	NO	P1 algorithm modules	Can't wire tools that don't exist yet
P3: Full PDF report	NO	P1 algorithms + P2 snapshot	Needs real findings to render
P4: FastAPI real endpoints	NO	P1 functions + P3 agent	Placeholder until both deliver modules
P4: Counterfactual UI	NO	P3 counterfactual.py	P3 must implement the endpoint first
P3: Paper Sections VI+	NO	P1 pseudocode + P2 eval data	Can't write algo section without P1 input
Everyone: Phase 2 start	NO	Sync 1 integration test passing	Hard gate — do not proceed if test fails
Everyone: Phase 3 start	NO	Sync 2 end-to-end test passing	Hard gate — code must be frozen

The key insight is that Weeks 1–6 are almost entirely parallel. All four people build independent modules simultaneously. The only hard constraint in that window is that P2 must give P1 and P3 the snapshot JSON by the end of Week 6 — that single handoff is the earliest possible integration point. Everything else can proceed independently until Sync 1.